TCP-Lab 讲义2.2：RDT-2.2

本讲义介绍 **RDT-2.2 协议**，它在 RDT-2.0 的基础上做了重要简化：**取消 NACK，只使用 ACK**，并通过**序号机制**区分新包和重复包。

---

1. RDT-2.2 是什么?

RDT-2.2 = RDT-2.0 的简化版本（只用 ACK，不用 NACK）

1.1 为什么要取消 NACK?

RDT-2.0 使用了两种响应报文：

- **ACK**：确认数据正确接收
- **NACK**：否认（数据损坏）

但实际上，**NACK 是冗余的**：

- 如果接收端能记住"上一次正确接收的包序号"
- 当收到损坏的包时，只需重复发送上一次的 ACK
- 发送端收到"重复的 ACK"就知道新包没被正确接收

核心思想：重复 ACK = NACK

1.2 RDT-2.2 vs RDT-2.0 对比

特性	RDT-2.0	RDT-2.2
响应类型	ACK + NACK	**只有 ACK**
序号机制	无（不判重）	**有（判重）**
数据损坏	发送 NACK(-1)	**发送重复 ACK**
重复包处理	无法处理	**识别并丢弃**
ACK 损坏	视为 NACK	**视为重复 ACK**

1.3 RDT-2.2 解决了什么问题?

1. **简化协议**：只需要一种响应报文（ACK），减少复杂度
2. **处理重复包**：通过序号区分新包和重复包
3. **为滑动窗口铺路**：序号机制是 GBN/SR 的基础

---

2. 序号机制详解

2.1 为什么需要序号?

考虑以下场景:

```
发送端 → [DATA seq=101] → 接收端
发送端 ← [ACK 101] ← 接收端（但ACK损坏）
发送端 → [重传 DATA seq=101] → 接收端
```

接收端第二次收到 seq=101，如何判断是新包还是重复包?

- **没有序号**：无法判断，可能重复交付数据
- **有序号**：比对期望序号，发现是重复包，丢弃

2.2 序号如何分配?

在本实验中:

- 每个数据包包含 100 个整数
- 序号按字节流编号：1, 101, 201, 301, ...
- 每发送一个包，序号增加 100

接收端维护 `expectedSeq`:

- 初始值：1
- 收到正确的 seq=1 后，`expectedSeq` = 101
- 收到正确的 seq=101 后，`expectedSeq` = 201
- 以此类推

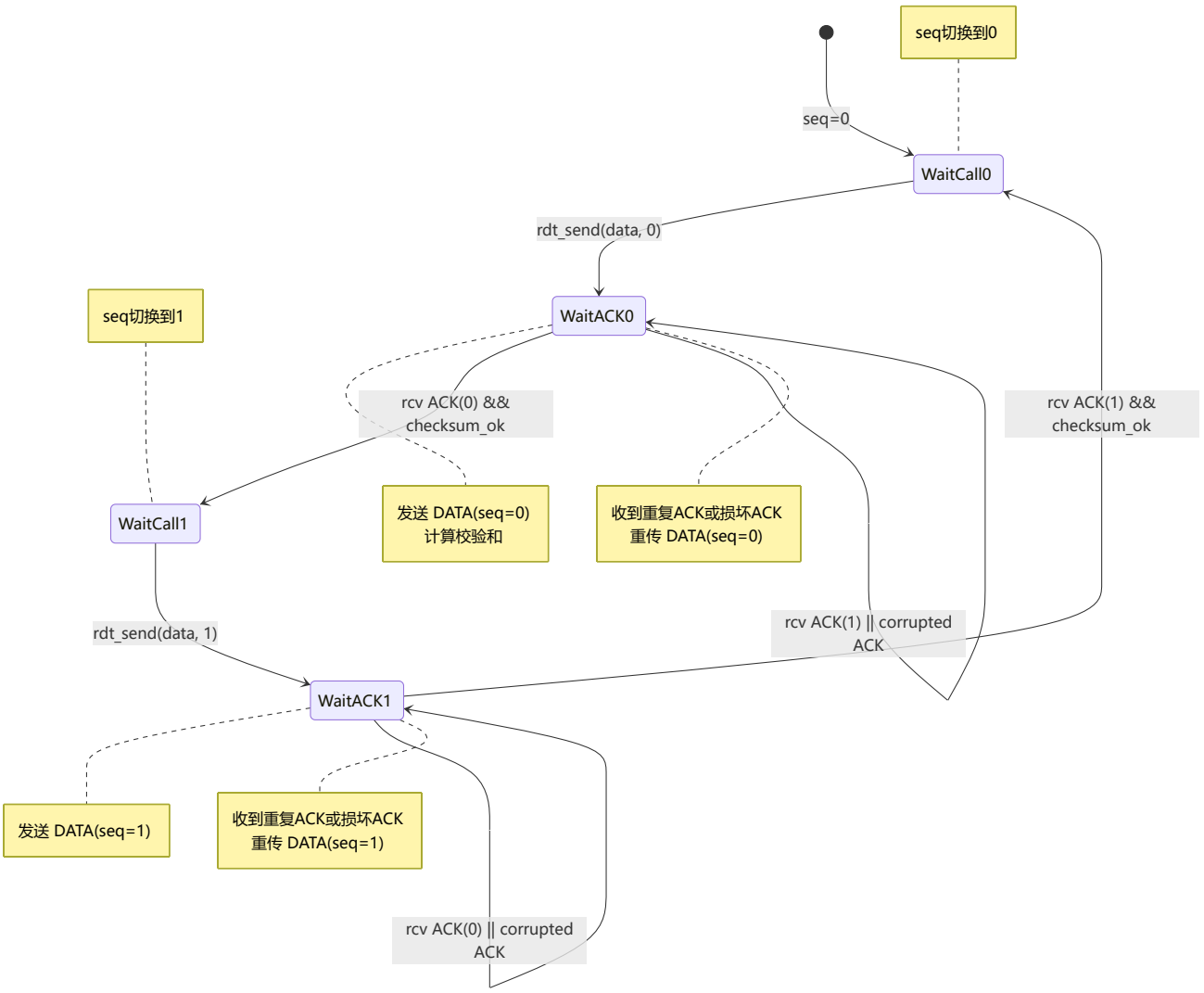
2.3 判重逻辑

```
if (recvSeq == expectedSeq) {
    // 新包，正确且有序
    交付数据;
    expectedSeq += 100;
    发送 ACK(recvSeq);
} else {
    // 重复包或失序包
    不交付数据;
    发送 ACK(lastAck);  // 重复ACK
}
```

---

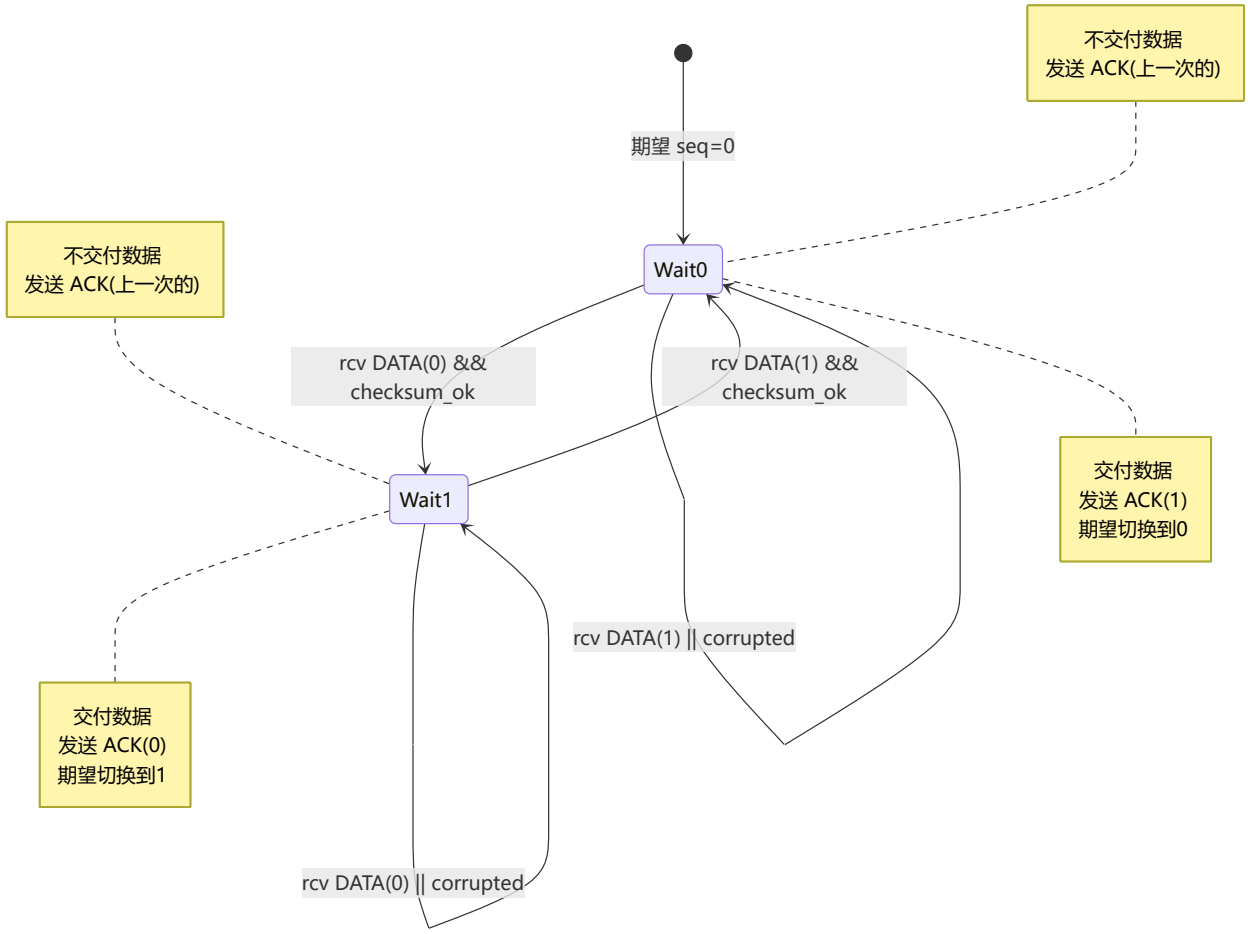
3. RDT-2.2 状态机模型

3.1 发送端状态机



注：理论上 RDT-2.2 只需要 0/1 两个序号交替即可，但本实验使用字节流序号（1, 101, 201...）

3.2 接收端状态机



4. RDT-2.2 实现详解

4.1 接收端关键变量

```
private int expectedSeq = 1;     // 期望接收的序号
private int lastAck = 0;          // 上一次发送的ACK序号
```

4.2 接收端核心逻辑（rdt_recv）

```
public void rdt_recv(TCP_PACKET recvPack) {
    int recvSeq = recvPack.getTcpH().getTh_seq();

    // 1. 检查校验和
    if (CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
        // 校验通过

        if (recvSeq == expectedSeq) {
            // 情况1：收到期望的包（新包）
            System.out.println("[RDT-2.2] Receive expected packet, seq: " + recvSeq);
```

```
            // 交付数据
            dataQueue.add(recvPack.getTcpS().getData());

            // 更新期望序号
            expectedSeq += 100;

            // 发送ACK确认这个包
            lastAck = recvSeq;
            tcpH.setTh_ack(recvSeq);

        } else {
            // 情况2：收到的不是期望的包（重复包或失序包）
            System.out.println("[RDT-2.2] Receive duplicate packet, seq: " + recvSeq);

            // 不交付数据，发送重复ACK
            tcpH.setTh_ack(lastAck);
        }

        // 构造并发送ACK
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        reply(ackPack);

    } else {
        // 情况3：校验失败（数据损坏）
        System.out.println("[RDT-2.2] Receive corrupted packet!");

        // RDT-2.2：不发送NACK，而是发送重复ACK
        tcpH.setTh_ack(lastAck);
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        reply(ackPack);
    }

    // 每20组交付一次
    if (dataQueue.size() == 20)
        deliver_data();
}
```

4.3 发送端核心逻辑（recv 和 waitACK）

recv 函数：处理接收到的 ACK

```
public void recv(TCP_PACKET recvPack) {
    // 检查ACK的校验和
    if (CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
        // 校验正确
        int ackNum = recvPack.getTcpH().getTh_ack();
        System.out.println("[RDT-2.2] Receive ACK: " + ackNum);
        ackQueue.add(ackNum);
    } else {
        // ACK损坏，视为重复ACK
```

```
            System.out.println("[RDT-2.2] Corrupted ACK, treat as duplicate");

            // 使用上一次的ACK值（如果队列不为空）
            if (!ackQueue.isEmpty()) {
                ackQueue.add(ackQueue.peek());
            } else {
                ackQueue.add(-1);  // 还没有收到过任何ACK
            }
        }

        waitACK();
    }
```

waitACK 函数：判断是否需要重传

```
public void waitACK() {
    if (!ackQueue.isEmpty()) {
        int currentAck = ackQueue.poll();
        int expectedAck = tcpPack.getTcpH().getTh_seq();

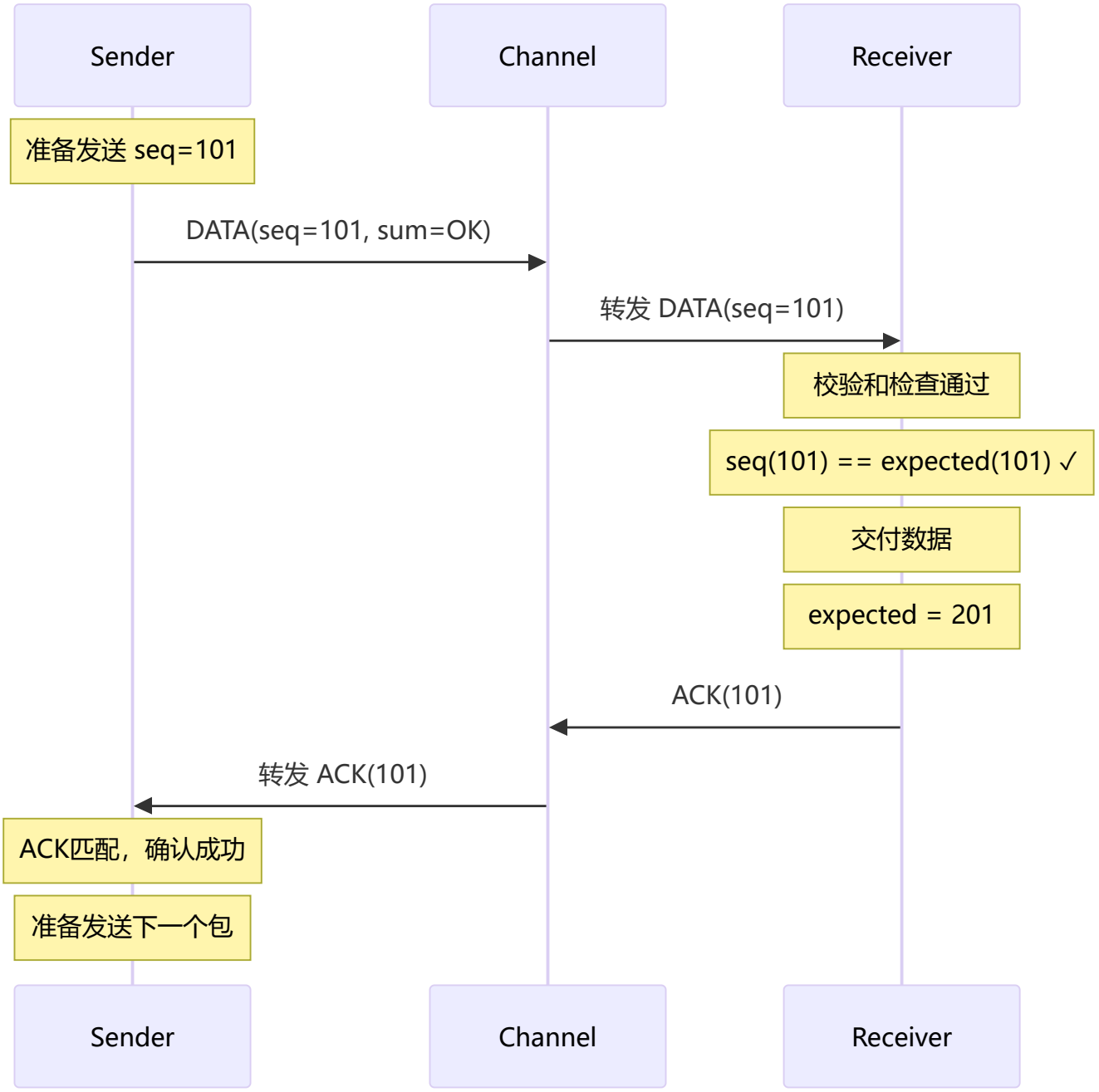
        if (currentAck == expectedAck) {
            // 收到正确的ACK
            System.out.println("[RDT-2.2] ACK matched, confirmed");
            flag = 1;  // 让 rdt_send 继续
        } else {
            // 收到重复ACK或无效ACK
            System.out.println("[RDT-2.2] Duplicate ACK, retransmit");
            udt_send(tcpPack);  // 重传当前包
            flag = 0;  // 保持等待
        }
    }
}
```

---

5. RDT-2.2 典型场景分析

5.1 场景1：正常传输

无错误情况下的正常数据传输流程。

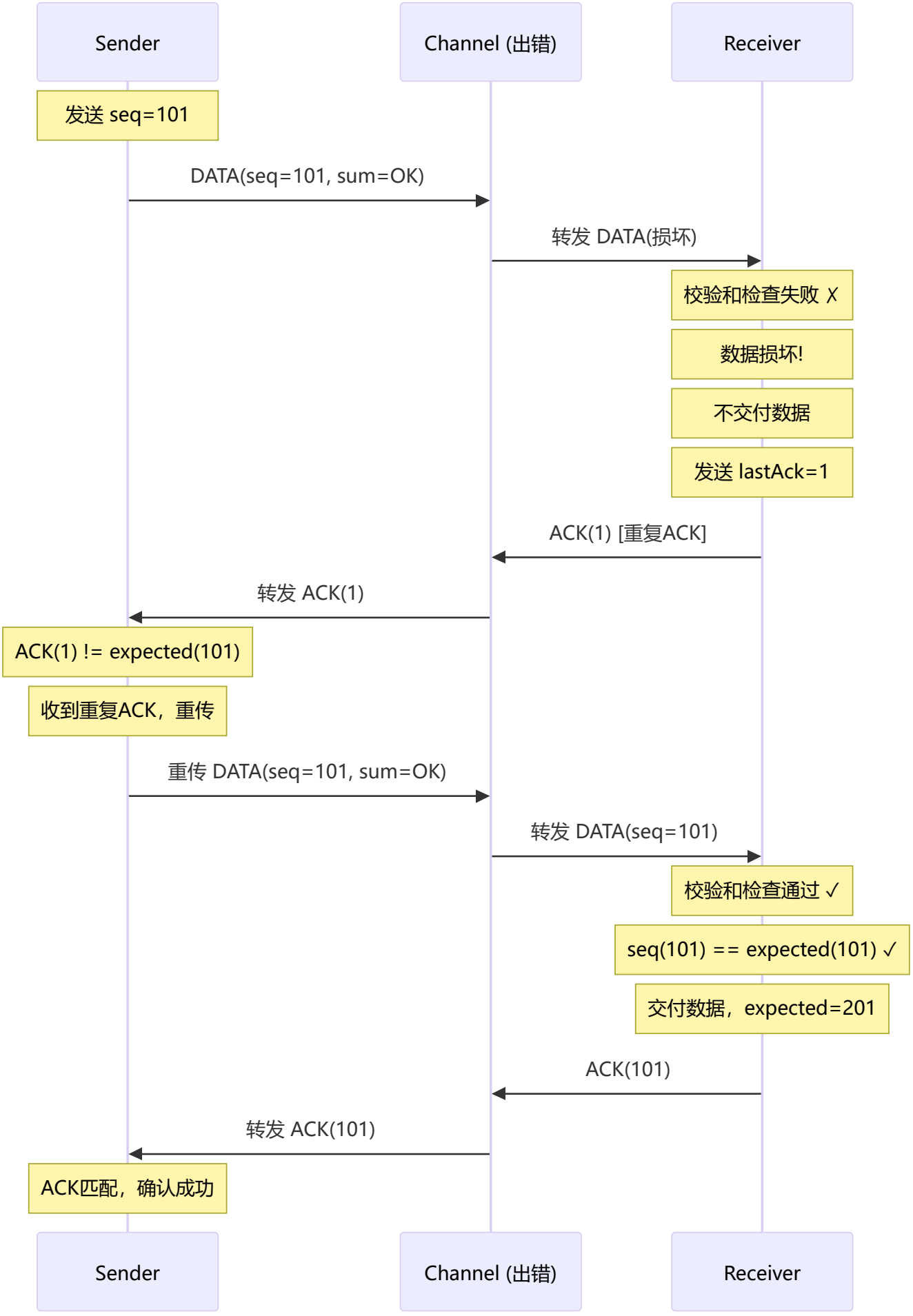


关键点：

- 接收端收到期望的序号，交付数据并更新期望序号
- 发送端收到匹配的ACK，可以发送下一个包

5.2 场景2：数据损坏

数据在传输过程中损坏，接收端发送重复ACK触发重传。

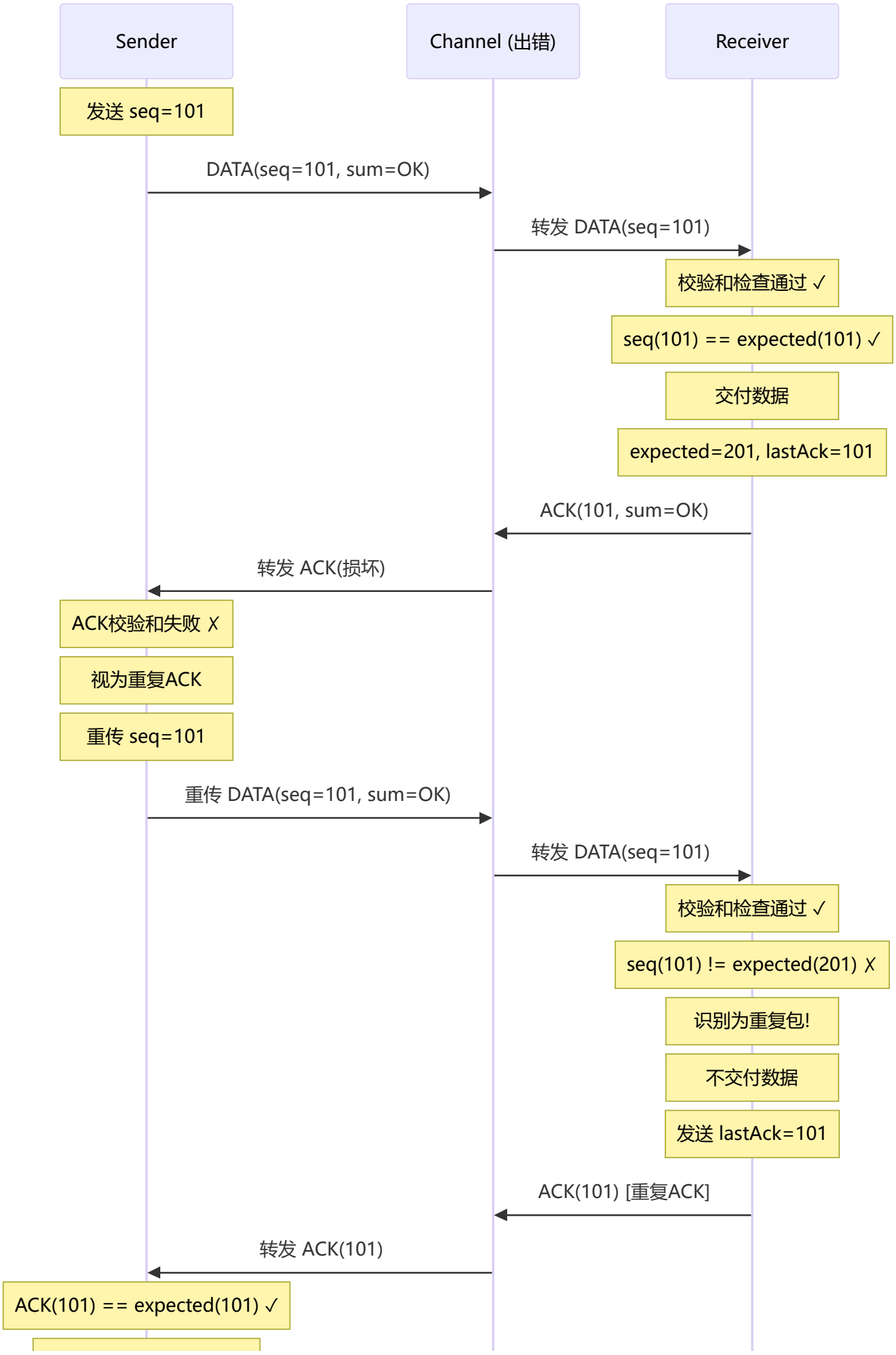

Sender
Channel (出错)
Receiver
发送 seq=101
DATA(seq=101, sum=OK)
转发 DATA(损坏)
校验和检查失败 ✗
数据损坏!
不交付数据
发送 lastAck=1
ACK(1) [重复ACK]
转发 ACK(1)
ACK(1) != expected(101)
收到重复ACK，重传
重传 DATA(seq=101, sum=OK)
转发 DATA(seq=101)
校验和检查通过 ✓
seq(101) == expected(101) ✓
交付数据，expected=201
ACK(101)
转发 ACK(101)
ACK匹配，确认成功
Sender
Channel (出错)
Receiver

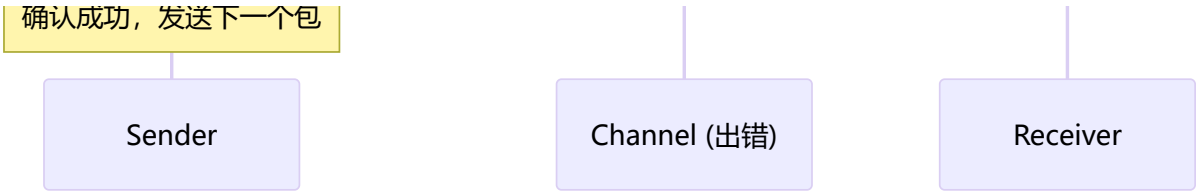
关键点：

- 接收端检测到数据损坏，发送重复ACK（上一次成功的ACK）
- 发送端识别重复ACK，触发重传
- 重传后数据正确，完成传输

5.3 场景3：ACK损坏

ACK在返回过程中损坏，导致发送端重传，接收端识别重复包。


Sender
Channel (出错)
Receiver
发送 seq=101
DATA(seq=101, sum=OK)
转发 DATA(seq=101)
校验和检查通过 ✓
seq(101) == expected(101) ✓
交付数据
expected=201, lastAck=101
ACK(101, sum=OK)
转发 ACK(损坏)
ACK校验和失败 ✗
视为重复ACK
重传 seq=101
重传 DATA(seq=101, sum=OK)
转发 DATA(seq=101)
校验和检查通过 ✓
seq(101) != expected(201) ✗
识别为重复包!
不交付数据
发送 lastAck=101
ACK(101) [重复ACK]
转发 ACK(101)
ACK(101) == expected(101) ✓

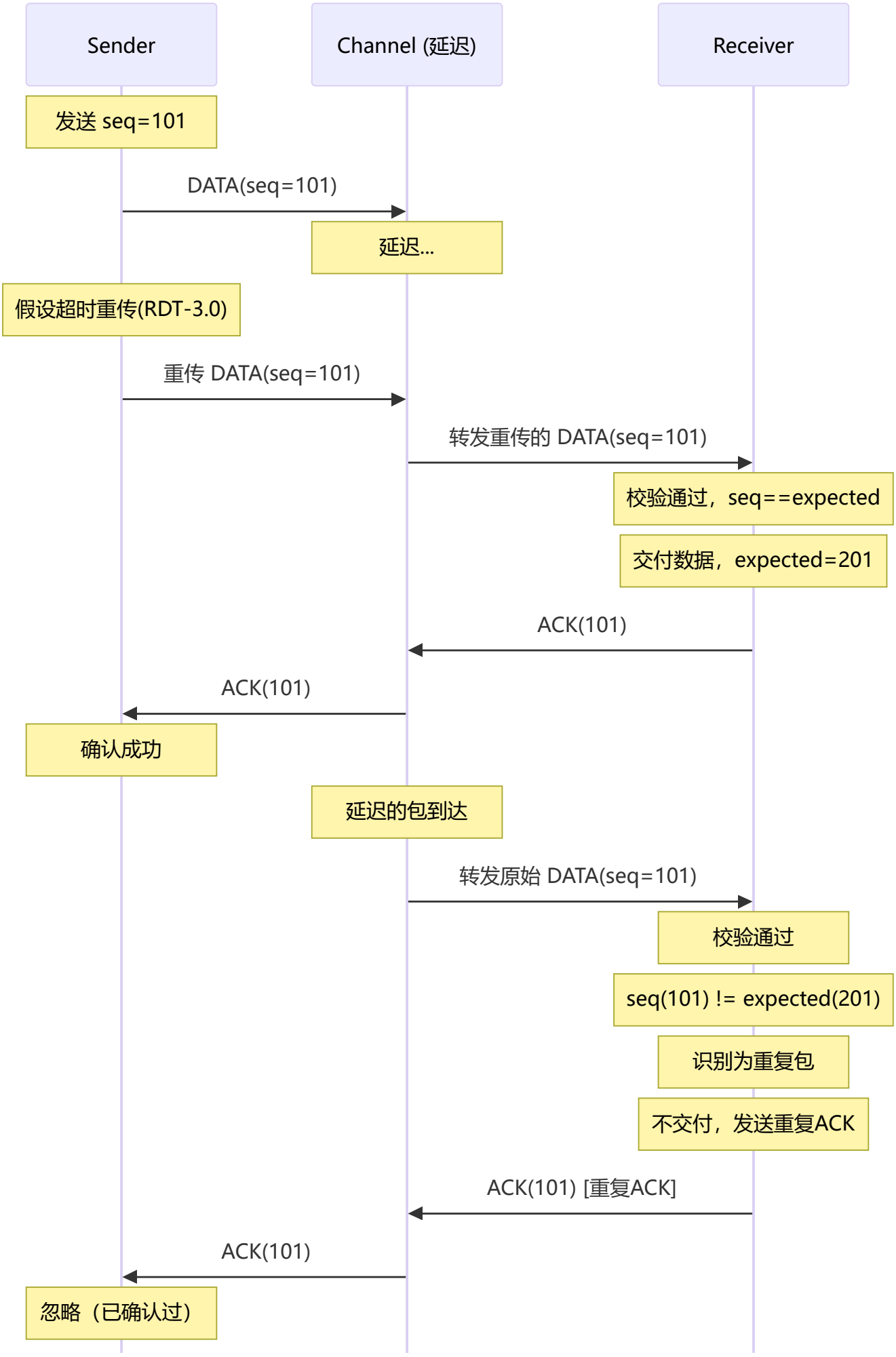


关键点:

- ACK损坏导致发送端重传
- 接收端通过序号判断识别重复包（seq=101 但 expected=201）
- 接收端不重复交付数据，避免数据重复
- 发送端最终收到正确ACK，继续传输

5.4 场景4：数据延迟导致的重复包（扩展）

虽然RDT-2.2不专门处理延迟，但序号机制天然支持识别重复。


Sender
Channel (延迟)
Receiver
发送 seq=101
DATA(seq=101)
延迟...
假设超时重传(RDT-3.0)
重传 DATA(seq=101)
转发重传的 DATA(seq=101)
校验通过，seq==expected
交付数据，expected=201
ACK(101)
ACK(101)
确认成功
延迟的包到达
转发原始 DATA(seq=101)
校验通过
seq(101) != expected(201)
识别为重复包
不交付，发送重复ACK
ACK(101) [重复ACK]
ACK(101)
忽略（已确认过）



关键点:

- 序号机制使接收端能够识别延迟的重复包
- 即使包延迟到达，也不会导致重复交付
- 这为RDT-3.0的超时重传机制打下基础

---

6. 运行与验证

6.1 验证步骤

1. **编译运行**:

```
mvn clean package
java -jar target/TCP_Test_final-1.0-SNAPSHOT.jar
```

2. **观察日志**:
   - 正常传输：`[RDT-2.2] Receive expected packet, seq: 101`
   - 数据损坏：`[RDT-2.2] Receive corrupted packet!` → 发送重复ACK → 重传
   - 重复包：`[RDT-2.2] Receive duplicate packet, seq: 101` → 不交付
   - ACK损坏：`[RDT-2.2] Corrupted ACK, treat as duplicate` → 重传
3. **检查输出**:
   - recvData.txt：无重复数据，行数正确
   - Log.txt：可以看到重传和重复ACK的记录

6.2 预期现象（eFlag=1）

✅ **正确性**:

- `recvData.txt` 包含完整数据，无错误值
- 不应出现重复数据（即使有重传）
- 行数应等于发送的总数据量

✅ **重传机制**:

- 数据损坏时触发重传
- ACK损坏时触发重传
- 收到重复ACK时触发重传

✅ **判重机制**:

- 接收端能识别重复包
- 重复包不会重复交付

- 但会回复重复ACK（帮助发送端尽快停止重传）

6.3 调试技巧

如何验证序号判重?

在接收端添加日志：

```
System.out.println("Received seq: " + recvSeq + ", Expected: " + expectedSeq);
```

观察是否出现 `recvSeq < expectedSeq` 的情况（重复包）

如何验证重复ACK?

在发送端添加日志：

```
System.out.println("Current ACK: " + currentAck + ", Expected ACK: " + expectedAck);
```

观察是否出现 `currentAck != expectedAck` 导致的重传

---

7. RDT-2.2 的改进与局限

7.1 相比 RDT-2.0 的改进

✅ **协议更简单**：

- 只有一种响应报文（ACK）
- 不需要特殊的 NACK 处理逻辑

✅ **能处理重复包**：

- 通过序号区分新包和重复包
- 避免重复交付数据

✅ **更符合实际**：

- 真实的 TCP 也不使用 NACK
- 序号机制是滑动窗口的基础

7.2 RDT-2.2 的局限性

❌ **不能处理丢包**：

- 如果数据包丢失，发送端永远等不到 ACK
- 如果 ACK 丢失，发送端也永远等不到

❌ **不能处理延迟**：

- 延迟的包可能导致混乱
- 需要超时机制

❌ **效率低（停等）**：

- 一次只能发一个包
- 链路利用率低

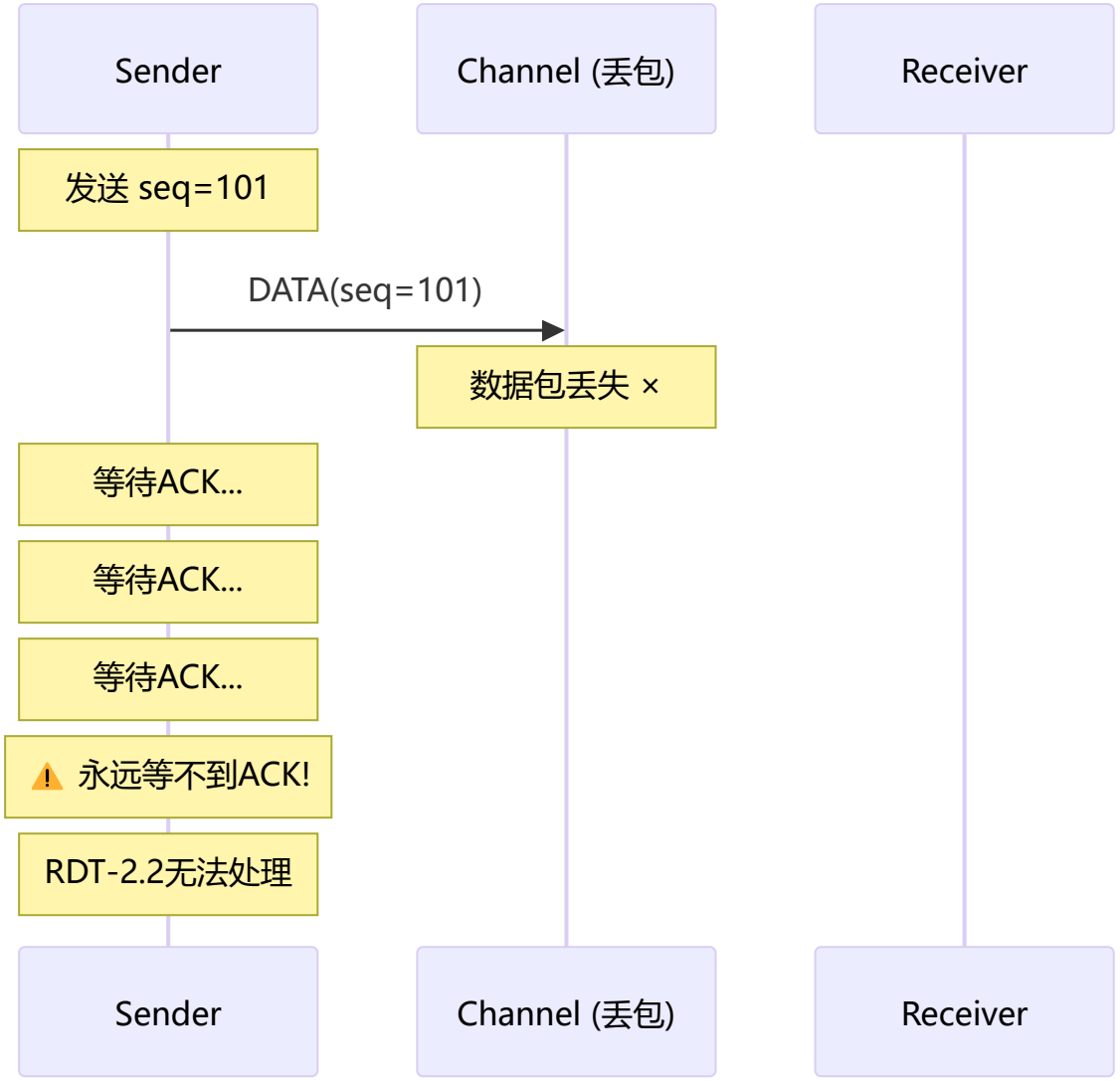
---

8. 从 RDT-2.2 到 RDT-3.0（预告）

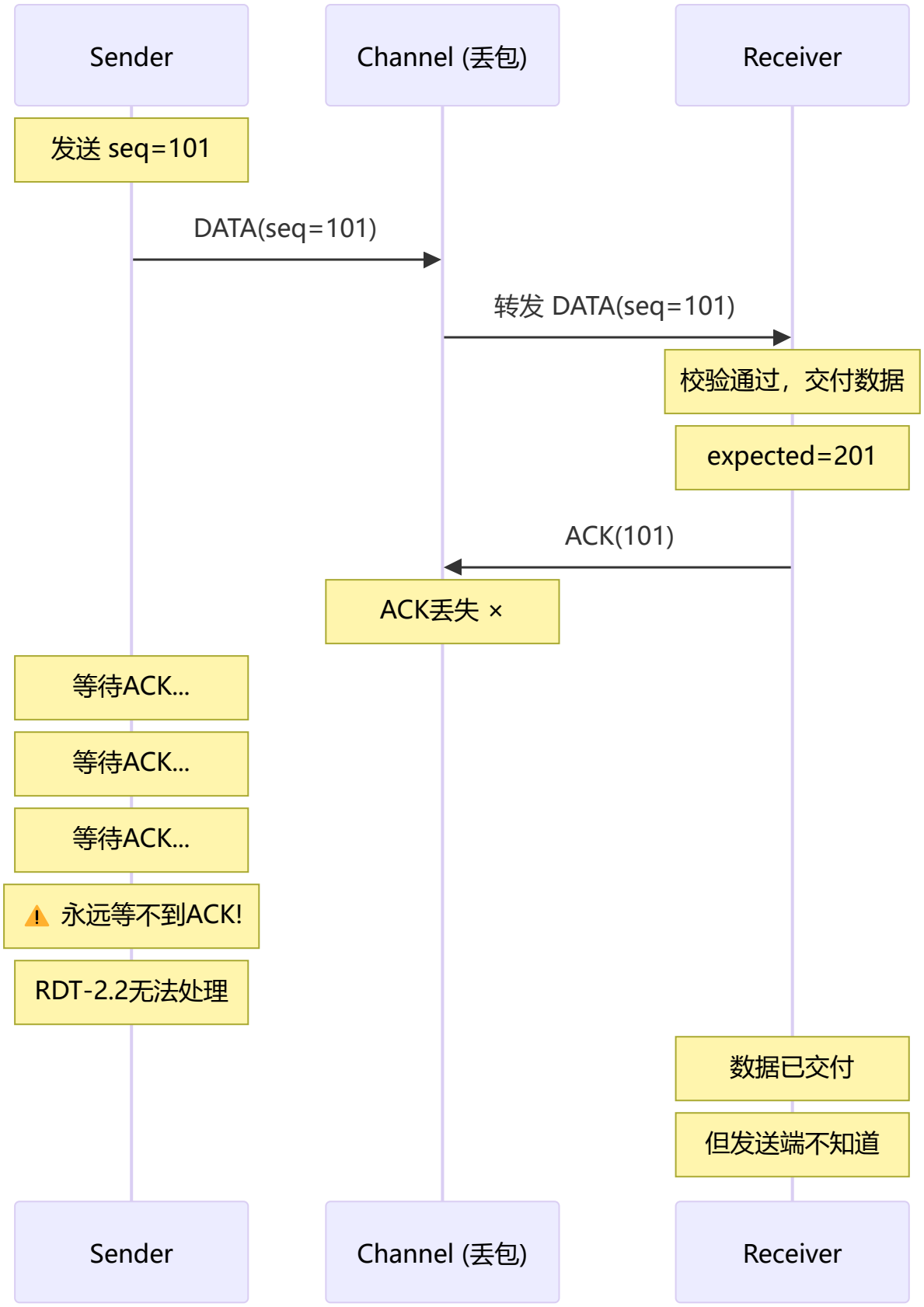
RDT-3.0 的核心改进：**超时重传**

8.1 超时重传的必要性

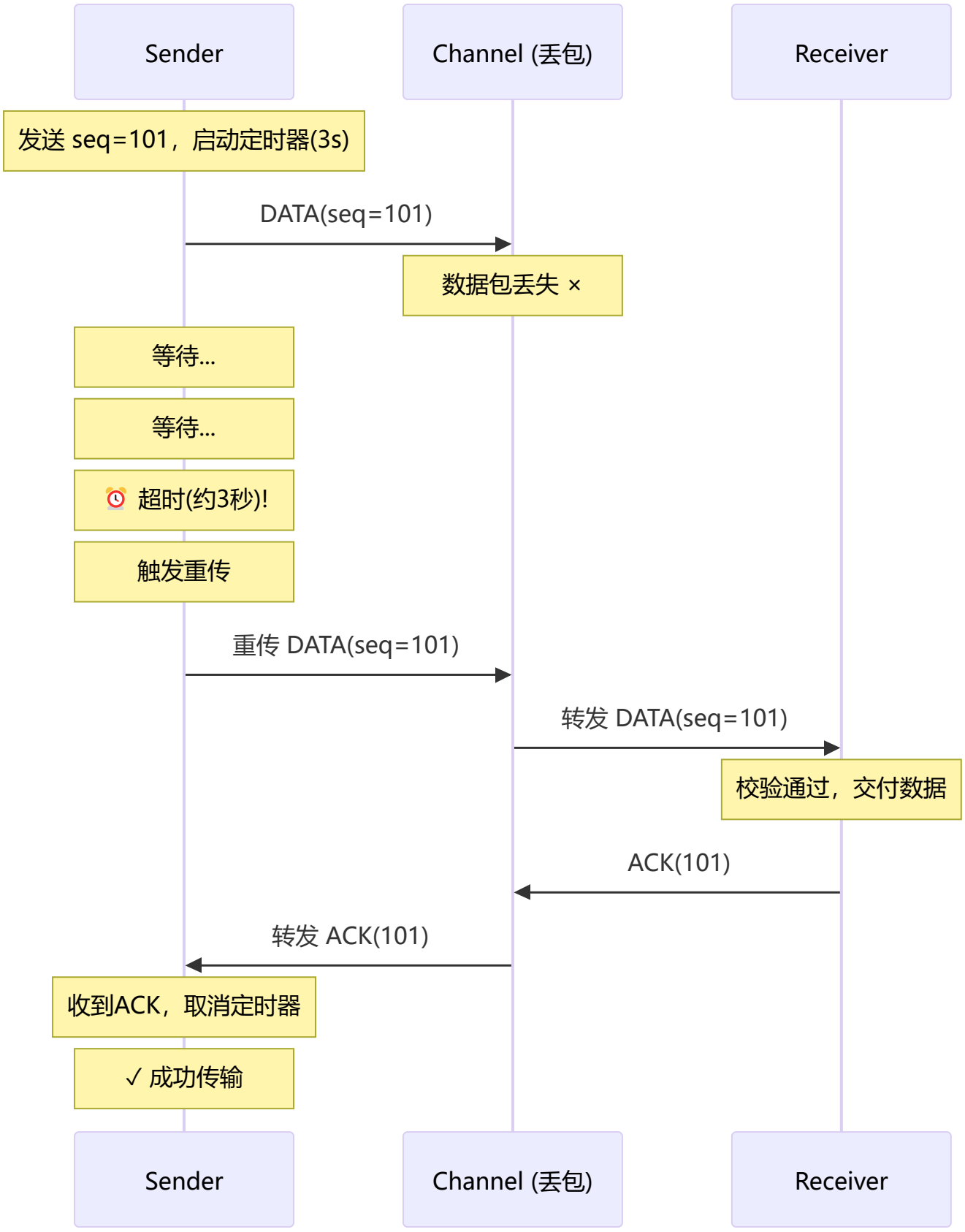
场景1：数据包丢失



场景2：ACK丢失


Sender
Channel (丢包)
Receiver
发送 seq=101
DATA(seq=101)
转发 DATA(seq=101)
校验通过，交付数据
expected=201
ACK(101)
ACK丢失 ×
等待ACK...
等待ACK...
等待ACK...
⚠ 永远等不到ACK!
RDT-2.2无法处理
数据已交付
但发送端不知道
Sender
Channel (丢包)
Receiver

RDT-3.0 的解决方案：超时重传



解决方案要点：

- ⏰ 发送包后启动定时器（约3秒）
- 🔄 如果超时还没收到 ACK，重传

- ✅ 收到正确 ACK 后取消定时器
- 🔁 结合RDT-2.2的序号机制处理重复包

8.2 RDT-3.0 新增内容

机制	作用
超时定时器	检测丢包或ACK丢失
超时重传	定时器超时时重传当前包
重复包处理	继承自RDT-2.2的序号机制

---

9. 总结

9.1 RDT-2.2 核心要点

方面	内容
目标	简化协议，只用ACK，处理重复包
核心机制	序号 + 重复ACK = NACK
序号作用	区分新包和重复包
重复ACK	数据损坏、ACK损坏、收到重复包
发送端	收到重复ACK时重传
接收端	收到重复包时不交付，发重复ACK
eFlag设置	1（只可能出错）
局限性	不处理丢包、延迟

9.2 实现检查清单

- [x] 接收端维护 `expectedSeq` 和 `lastAck`
- [x] 接收端根据序号判断新包/重复包
- [x] 接收端收到重复包时不交付，发重复ACK
- [x] 接收端数据损坏时发重复ACK（不发NACK）
- [x] 发送端ACK损坏时视为重复ACK
- [x] 发送端收到重复ACK时重传
- [x] 测试 `eFlag=1` 下正确检测错误和重复
- [x] 验证 `recvData.txt` 无重复数据

9.3 关键代码对照表

文件	函数	RDT-2.0	RDT-2.2 变化
TCP_Receiver.java	成员变量	`sequence`	`expectedSeq` + `lastAck`
TCP_Receiver.java	rdt_recv	损坏发NACK	**判断序号，损坏发重复ACK**
TCP_Sender.java	recv	ACK损坏用-1	**ACK损坏视为重复ACK**
TCP_Sender.java	waitACK	收到-1重传	**收到重复ACK重传**

9.4 下一步

继续学习 **RDT-3.0**（加入超时重传机制，解决丢包和延迟问题）。

附录：理论上的0/1序号

教科书中的 RDT-2.2 通常使用 **0 和 1 交替作为序号**，因为停等协议一次只有一个包在飞行中，两个序号足够区分"当前包"和"上一个包"。

但本实验使用**字节流序号**（1, 101, 201...），这更接近真实 TCP 的实现，也为后续的滑动窗口协议铺路。

两种方式的本质是一样的：**用序号区分新包和重复包。**
